Jenkins Pipeline and Real Deployment Guide

A step-by-step guide to this project's current Jenkinsfile, Docker usage, and real QA/production deployment workflows.

1. Current Pipeline Flow

Step	What Jenkins Does
1. Checkout	Downloads the branch that triggered the build.
2. Build	Compiles and packages the Maven project without running tests.
3. QA Deploy	Currently a placeholder; it only prints a message.
4. Regression	Runs headless TestNG UI and API tests using Jenkins credentials.
5. Allure	Publishes the test execution report from allure-results.
6. Qodana	Runs static analysis in a temporary Docker container.

Important: the current Deploy to QA stage does not deploy an application. It contains only echo 'deploy to qa'. The pipeline currently builds, tests, reports, and analyzes this automation framework.

2. Current Jenkinsfile, Step by Step

1. Jenkins Agent and Maven

agent any lets Jenkins select an available build machine. The tools block loads the Maven installation named MAVEN from Manage Jenkins -> Tools. In a company, the build machine is commonly a Linux VM, cloud runner, or temporary Kubernetes agent.

```
agent any
tools { maven 'MAVEN' }
```

2. Checkout and Build

checkout scm retrieves the Git branch that triggered the pipeline. mvn -DskipTests clean package removes old output, compiles the Java project, downloads Maven dependencies, and creates the build package. Jenkins then archives matching JAR files from target/*.jar.

```
checkout scm
mvn -DskipTests clean package
```

3. Regression Automation

Jenkins retrieves the OpenCart login and GoRest bearer token from Jenkins Credentials, then passes them as temporary environment variables to Maven. Headless mode allows Playwright browser tests to run on a server without a visible desktop.

```
mvn clean test \
-Dsurefire.suiteXmlFiles=src/test/resources/TestRunners/testng_regression.xml \
-Dheadless=true \
-Dusername="$OC_USERNAME" \
-Dpassword="$OC_PASSWORD" \
-Dgorest.bearer.token="$GOREST_TOKEN"
```

catchError keeps the overall Jenkins build green even when a test fails, while marking the regression stage as failed. This permits Allure reporting. For a production release gate, critical test failures should normally block deployment.

4. Allure and Qodana

The Allure plugin reads the allure-results folder and publishes the report in Jenkins. Qodana runs inside jetbrains/qodana-jvm-community:2026.2 on the Jenkins agent. The project folder is mounted into the container, Qodana scans the code, writes qodana-results, and exits. It is a temporary code-analysis container, not the deployed application.

3. Local Docker vs Real Deployment

Local development	Real project deployment
Laptop -> docker build -> docker run	Jenkins -> build image -> push registry -> platform pulls and starts it
Container runs on developer laptop	Container runs on QA/staging/prod infrastructure
Often manual and short-lived	Versioned, monitored, access-controlled, and repeatable

The private registry can be Docker Hub, AWS ECR, Azure Container Registry, Google Artifact Registry, JFrog Artifactory, or GitLab Container Registry. Jenkins usually does not host the deployed application; it builds and delivers a versioned image to the platform that does.

4. Where Docker Runs in Production

A. Docker on a QA or Production Server

Jenkins connects to a Linux server with an SSH key. Docker is installed on that remote server, which pulls the approved image and runs it.

```
docker pull registry.company.com/my-app:125
docker stop my-app || true
docker rm my-app || true
docker run -d --name my-app -p 8080:8080 registry.company.com/my-app:125
```

B. Docker Compose

For several services on one server, Docker Compose pulls the current images and starts them together.

```
docker compose pull
docker compose up -d
```

C. Kubernetes

With Kubernetes, Jenkins updates the image reference. Kubernetes worker nodes pull the image and create containers. The team normally uses kubectl or deployment tools rather than docker run directly.

```
kubectl set image deployment/my-app \
my-app=registry.company.com/my-app:125 \
--namespace qa
kubectl rollout status deployment/my-app --namespace qa
```

D. Managed Container Platforms

AWS ECS/Fargate, Azure Container Apps, Google Cloud Run, and similar cloud products start the containers for the team. Jenkins updates the image version or service definition, then the cloud platform performs the rollout.

5. Typical Real QA and Production Flow

Step	Delivery Activity
1	Developer opens a pull request. CI runs compile, unit tests, API/UI tests, and static analysis.
2	Review is approved and code merges to the main branch.
3	Jenkins builds one immutable image, for example my-app:1.8.0-125, and pushes it to the private registry.
4	Jenkins deploys that image to QA. A health check and smoke/regression suite run against the QA environment.
5	A quality gate or manual approval authorizes production deployment.
6	Jenkins deploys the same already-tested image tag to production.
7	Health checks, logs, monitoring, and alerts verify the rollout. A failed rollout is rolled back to the previous stable image.

6. Example Production Commands

Build and push a versioned image

```
docker build -t registry.company.com/my-app:${BUILD_NUMBER} .
docker push registry.company.com/my-app:${BUILD_NUMBER}
```

Deploy to QA and verify health

```
kubectl set image deployment/my-app \
my-app=registry.company.com/my-app:${BUILD_NUMBER} \
--namespace qa
kubectl rollout status deployment/my-app --namespace qa
curl --fail --retry 10 --retry-delay 5 https://qa.example.com/actuator/health
```

Require approval before production

```
input message: 'Approve production deployment?', ok: 'Deploy'
```

7. Production Standards

- Keep all passwords, tokens, SSH keys, cloud credentials, and registry credentials in Jenkins Credentials or a secret manager.

- Use separate configurations and credentials for QA, staging, and production.

- Build once and promote the same immutable image tag from QA to production. Do not deploy latest.

- Run health checks and smoke tests after deployment; roll back when the new version is unhealthy.

- Archive test evidence, deployment logs, image tags, Allure reports, and static-analysis reports.

- Use quality gates or a manual approval before production when required by the release process.

8. Recommended Next Step for This Project

Keep the current Jenkinsfile as the automation-test pipeline. When the application team provides a QA deployment target, replace the Deploy to QA placeholder with the correct integration for its hosting model: SSH/Docker Compose, Kubernetes, or a managed cloud container service. Add a QA health-check stage before the regression suite runs.